

LESSON 9: ADVANCED REACT & STATE MANAGEMENT

Global Architecture, Context API, Redux Toolkit, Performance Optimization & Code Splitting

1. THE PROBLEM OF PROP DRILLING & GLOBAL STATE ARCHITECTURE

In standard React architectures, state moves downward via props. When an application grows, widely separated components often require access to the exact same information (e.g., authenticated user profiles, theme tokens, shopping cart contents, or feature flags).

1.1 What is Prop Drilling?

Prop drilling occurs when data must be passed through intermediate components that have no operational need for that data, solely to deliver it to a deeply nested descendant.

[App Component] (Holds: user, theme)

|

▼ (passes: user, theme)

[DashboardLayout] (Does NOT need user/theme)

|

▼ (passes: user, theme)

[MainContentArea] (Does NOT need user/theme)

|

▼ (passes: user, theme)

[UserProfileCard] ◀— FINALLY CONSUMES: user, theme

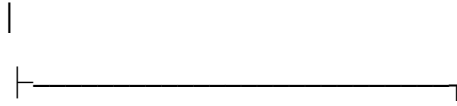
Consequences of Prop Drilling:

- **Fragile Refactoring:** Changing a prop signature requires manually updating every intermediary component.
- **Component Pollution:** Components become bloated with pass-through props, violating Single Responsibility principles.
- **Unnecessary Rerenders:** Passing down new object references through components can trigger cascading rerenders down the branch.

2. THE CONTEXT API & useContext

The React Context API creates a communication channel directly between an ancestor Provider and any consuming descendant, bypassing intermediate props entirely.

[ThemeContext.Provider value={theme}]



[Intermediate A]

[Intermediate B]



[Consumer Node]

[Consumer Node]

useContext(ThemeContext) useContext(ThemeContext)

2.1 The 3-Step Context Pattern

1. **Creation:** Instantiate the context token using `createContext()`.
2. **Provision:** Wrap the parent tree with `<Context.Provider value="{...}">`.
3. **Consumption:** Consume the provided value in any descendant via the `useContext()` hook.

JavaScript

```
import { createContext, useContext, useState } from "react";
```

```
// 1. CREATE: Initialize Context with a safe default value
```

```
const AuthContext = createContext(null);
```

```
// 2. PROVIDE: Wrap children with Provider in an ancestor
```

```
export function AuthProvider({ children }) {
```

```
  const [currentUser, setCurrentUser] = useState({ name: "Alex", role: "admin" });
```

```
  const logout = () => setCurrentUser(null);
```

```
return (  
  <AuthContext.Provider value={{ currentUser, logout }}>  
    {children}  
  </AuthContext.Provider>  
);  
}
```

// 3. CONSUME: Access value directly via useContext

```
function UserBadge() {  
  const { currentUser, logout } = useContext(AuthContext);  
  
  if (!currentUser) return <button>Sign In</button>;  
  
  return (  
    <div>  
      <span>Signed in as: {currentUser.name}</span>  
      <button onClick={logout}>Log Out</button>  
    </div>  
  );  
}
```

3. COMPLEX LOCAL STATE: THE useReducer HOOK

While useState works well for simple primitives, it becomes unwieldy when state transitions involve multiple interdependent fields or when the next state depends heavily on the previous state.

3.1 What is a Reducer?

A reducer is a pure mathematical function that takes the current state and an incoming action object, returning the next immutable state snapshot:

Reducer: (State,Action) → New State

- **State:** The immutable current memory snapshot.
- **Action:** A plain JavaScript object describing an event, typically structured with a type string and an optional payload.
- **Dispatch:** The trigger function that dispatches an action to the reducer.

3.2 Anatomy of useReducer

JavaScript

```
import { useReducer } from "react";
```

```
// 1. Define explicit initial state
```

```
const initialCartState = {  
  items: [],  
  totalAmount: 0,  
};
```

```
// 2. Define pure reducer function (No side effects, purely returns new state)
```

```
function cartReducer(state, action) {  
  switch (action.type) {  
    case "cart/addItem": {  
      const updatedItems = [...state.items, action.payload];  
      const updatedTotal = updatedItems.reduce((sum, item) => sum + item.price, 0);  
      return { items: updatedItems, totalAmount: updatedTotal };  
    }  
    case "cart/removeItem": {  
      const updatedItems = state.items.filter((item) => item.id !== action.payload.id);
```

```
    const updatedTotal = updatedItems.reduce((sum, item) => sum + item.price, 0);  
    return { items: updatedItems, totalAmount: updatedTotal };  
  }  
  case "cart/reset":  
    return initialCartState;  
  default:  
    return state;  
  }  
}
```

// 3. Component integration

```
export default function ShoppingCart() {  
  const [state, dispatch] = useReducer(cartReducer, initialCartState);  
  
  const handleAdd = () => {  
    dispatch({  
      type: "cart/addItem",  
      payload: { id: "item-101", name: "Mechanical Keyboard", price: 120 },  
    });  
  };  
  
  return (  
    <div>  
      <h3>Cart Total: ${state.totalAmount}</h3>  
      <button onClick={handleAdd}>Add Keyboard ($120)</button>  
      <button onClick={() => dispatch({ type: "cart/reset" })}>Clear Cart</button>  
    </div>  
  );  
}
```

```

<ul>

  {state.items.map((i, index) => (
    <li key={index}>{i.name} — ${i.price}</li>
  ))}

</ul>

</div>

);
}

```

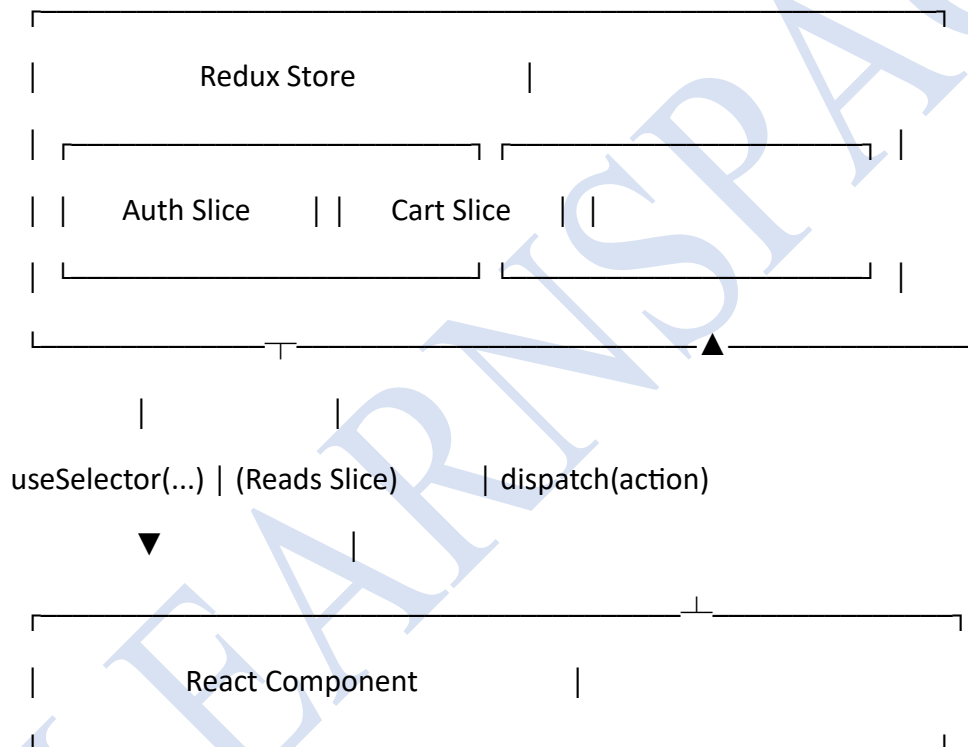
4. ARCHITECTURAL COMPARISON: CONTEXT API VS REDUX

Dimension	React Context API	Redux Toolkit (RTK)
Primary Purpose	Dependency injection & prop-drilling avoidance.	Predictable, centralized application state management.
Setup & Boilerplate	Native to React, zero external installation.	Requires @reduxjs/toolkit and react-redux.
State Mutation Model	Context provides values; updates rely on useState/useReducer.	Strict dispatch-action-reducer unidirectional pipeline.
Rerender Behavior	All components reading a context rerender when its value updates.	Granular: Components rerender <i>only</i> if their selected slice changes.
Asynchronous Side Effects	Manual handling inside useEffect or callbacks.	Built-in async pipelines using createAsyncThunk / RTK Query.
DevTools Ecosystem	Standard React DevTools inspector.	Redux DevTools: Time-travel debugging, action replays.

Dimension	React Context API	Redux Toolkit (RTK)
Best Used For	Static or low-frequency data: Themes, locale, user auth session.	High-frequency, large-scale data: Carts, feeds, complex caching.

5. REDUX TOOLKIT (RTK): MODERN REDUX ARCHITECTURE

Legacy Redux required significant boilerplate: manual action creators, immutable spread operators, action type constants, and complex store wiring. **Redux Toolkit (RTK)** is the modern standard for writing Redux logic efficiently.



5.1 Core Redux Concepts

- **Store:** The single source of truth containing the global state tree.
- **Slice:** A domain-specific module (created via `createSlice`) containing initial state, reducers, and automatically generated action creators.
- **Reducers & Immer:** RTK uses the **Immer** library under the hood. You can write "mutative" code (e.g., `state.count += 1`), and Immer safely translates it into an immutable state update.

- **Actions:** Objects containing a type string and data payload dispatched to notify the store of an event.
- **Selectors:** Functions used by useSelector to extract and memoize specific slices of store data.

6. REDUX TOOLKIT IN PRACTICE: STEP-BY-STEP INTEGRATION

Step 1: Create a Slice (cartSlice.js)

JavaScript

```
import { createSlice } from "@reduxjs/toolkit";
```

```
const initialState = {
```

```
  items: [],
```

```
  totalPrice: 0,
```

```
};
```

```
export const cartSlice = createSlice({
```

```
  name: "cart",
```

```
  initialState,
```

```
  reducers: {
```

```
    addItem: (state, action) => {
```

```
      // Immer allows direct mutations safely under the hood
```

```
      state.items.push(action.payload);
```

```
      state.totalPrice += action.payload.price;
```

```
    },
```

```
    removeItem: (state, action) => {
```

```
      const index = state.items.findIndex((item) => item.id === action.payload);
```

```
      if (index !== -1) {
```

```
        state.totalPrice -= state.items[index].price;
```



```
    state.items.splice(index, 1);
  }
},
clearCart: (state) => {
  state.items = [];
  state.totalPrice = 0;
},
},
});

// Auto-generated action creators
export const { addItem, removeItem, clearCart } = cartSlice.actions;

// Selectors
export const selectCartItems = (state) => state.cart.items;
export const selectCartTotal = (state) => state.cart.totalPrice;

export default cartSlice.reducer;
```

Step 2: Configure the Global Store (store.js)

JavaScript

```
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "../cartSlice";

export const store = configureStore({
  reducer: {
    cart: cartReducer,
```

```
},  
});
```

Step 3: Provide the Store to React (index.js or App.js)

JavaScript

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import { Provider } from "react-redux";  
import { store } from "../store";  
import App from "../App";  
  
const root = ReactDOM.createRoot(document.getElementById("root"));  
root.render(  
  <Provider store={store}>  
    <App />  
  </Provider>  
);
```

Step 4: Dispatch Actions and Select State in Components

JavaScript

```
import { useSelector, useDispatch } from "react-redux";  
import { addItem, removeItem, selectCartItems, selectCartTotal } from "../cartSlice";  
  
export default function CartView() {  
  const dispatch = useDispatch();  
  const items = useSelector(selectCartItems);  
  const total = useSelector(selectCartTotal);
```

```

const handleAddProduct = () => {
  dispatch(
    addItem({
      id: "prod-" + Date.now(),
      name: "Wireless Mouse",
      price: 49.99,
    })
  );
};

return (
  <div>
    <h2>Order Review</h2>
    <p>Cart Total: <strong>${total.toFixed(2)}</strong></p>
    <button onClick={handleAddProduct}>Add Mouse ($49.99)</button>

    <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} — ${item.price}
          <button onClick={() => dispatch(removeItem(item.id))}>Delete</button>
        </li>
      ))}
    </ul>
  </div>
);

```

```
}
```

7. ASYNC THUNKS: HANDLING ASYNCHRONOUS DATA IN REDUX

Standard reducers must remain synchronous, pure functions. Asynchronous operations—such as calling REST APIs—are managed using **Async Thunks** via `createAsyncThunk`.

Lifecycle of an Async Thunk

When dispatched, an async thunk automatically emits three lifecycle action types:

1. **pending:** Emitted when the async call starts (used to trigger loading spinners).
2. **fulfilled:** Emitted when the promise resolves (carries the returned data in `action.payload`).
3. **rejected:** Emitted if the network request fails (carries error details in `action.error`).

JavaScript

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
```

```
// 1. Create the async thunk
```

```
export const fetchCourses = createAsyncThunk(
  "courses/fetchCourses",
  async (_, { rejectWithValue }) => {
    try {
      const response = await fetch("https://jsonplaceholder.typicode.com/posts?_limit=5");
      if (!response.ok) throw new Error("Network response failed");
      return await response.json();
    } catch (err) {
      return rejectWithValue(err.message);
    }
  }
);
```

```
// 2. Handle the thunk lifecycle in extraReducers

export const courseSlice = createSlice({
  name: "courses",
  initialState: {
    data: [],
    loading: false,
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchCourses.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchCourses.fulfilled, (state, action) => {
        state.loading = false;
        state.data = action.payload;
      })
      .addCase(fetchCourses.rejected, (state, action) => {
        state.loading = false;
        state.error = action.payload || "Could not load courses";
      });
  },
});
```

```
export default courseSlice.reducer;
```

8. ADVANCED CUSTOM HOOK PATTERNS

Custom hooks encapsulate stateful logic, side effects, and subscription management into isolated, reusable abstractions.

Example: Reusable `useLocalStorage` Hook

A production-ready custom hook that synchronizes a React state variable directly with the browser's `localStorage`:

JavaScript

```
import { useState, useEffect } from "react";

export function useLocalStorage(key, initialValue) {
  // 1. Lazy state initialization to read from storage only on mount
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.error(`Error reading key "${key}" from localStorage:`, error);
      return initialValue;
    }
  });

  // 2. Sync state changes to localStorage
  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      console.error(`Error writing key "${key}" to localStorage:`, error);
    }
  });
}
```

```
    console.error(`Error writing key "${key}" to localStorage:`, error);
  }
}, [key, storedValue]);

return [storedValue, setStoredValue];
}
```

9. REACT PERFORMANCE OPTIMIZATION

React's default behavior is to rerender a component whenever its parent rerenders, regardless of whether its props have changed. While usually fast, unoptimized rerenders in complex trees can degrade performance.

9.1 Component Memoization with React.memo

React.memo is a higher-order component that wraps a component, performing a **shallow comparison** of its props. If the props are identical to the previous render, React skips rendering that component and reuses the last rendered output.

JavaScript

```
import React from "react";

// Child will NOT rerender if 'name' and 'badge' remain referentially equal
export const UserCard = React.memo(function UserCard({ name, badge }) {
  console.log("UserCard rendered!");
  return (
    <div className="card">
      <h4>{name}</h4>
      <span>{badge}</span>
    </div>
  );
});
```

9.2 Memoizing Values & Callbacks (useMemo and useCallback)

- **useMemo:** Caches the result of an expensive calculation across renders.
- **useCallback:** Preserves the reference identity of a function passed down as a prop, preventing React.memo wrapped child components from breaking their shallow comparison checks.

10. CODE SPLITTING & LAZY LOADING WITH React.lazy & <Suspense>

By default, bundlers like Webpack and Vite pack an entire React application into a single JavaScript bundle. On the initial page load, the browser must download, parse, and execute this entire file before displaying anything.

Code Splitting splits this bundle into smaller chunks that load on demand, dramatically reducing initial page load times.

Monolithic Bundle:

[Home + Dashboard + Admin + Settings + Analytics] —► 2.8 MB (Slow initial load)

Code-Split Bundles:

[Main Chunk (Home)] —► 320 KB (Loads instantly!)

|

└─► User navigates to /admin —► [Admin Chunk] (Loads 450 KB on demand)

10.1 Implementing Route-Based Lazy Loading

Using `React.lazy()` with dynamic imports (`import()`) and wrapping routes inside `<Suspense fallback="{<Spinner/>}">`:

JavaScript

```
import React, { Suspense, lazy } from "react";
```

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
```

```
// Dynamic asynchronous imports
```

```
const Home = lazy(() => import("./routes/Home"));
```

```
const AnalyticsDashboard = lazy(() => import("./routes/AnalyticsDashboard"));
```



```

const AdminPanel = lazy(() => import("./routes/AdminPanel"));

function LoadingFallback() {
  return <div style={{ padding: 24, textAlign: "center" }}>Loading module...</div>;
}

export default function App() {
  return (
    <BrowserRouter>
      <nav style={{ display: "flex", gap: 12, padding: 12 }}>
        <Link to="/">Home</Link>
        <Link to="/analytics">Analytics</Link>
        <Link to="/admin">Admin</Link>
      </nav>

      { /* Suspense catches lazy components and shows fallback during network download */ }

      <Suspense fallback={<LoadingFallback />}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/analytics" element={<AnalyticsDashboard />} />
          <Route path="/admin" element={<AdminPanel />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}

```

11. COMPLETE HYBRID STATE ARCHITECTURE CASE STUDY

Below is an enterprise-grade component combining:

- The **Context API** (managing global user theme preferences).
- **useReducer** (managing multi-step localized form transitions).
- Memoization hooks (useMemo, useCallback) ensuring optimal performance.

JavaScript

```
import React, { createContext, useContext, useReducer, useMemo, useCallback } from "react";
```

```
// =====
```

```
// 1. CONTEXT LAYER: Theme Preference Provider
```

```
// =====
```

```
const ThemeContext = createContext();
```

```
export function ThemeProvider({ children }) {
```

```
  const [isDark, setIsDark] = useReducer((prev) => !prev, false);
```

```
  const contextValue = useMemo(() => ({ isDark, toggleTheme: setIsDark }), [isDark]);
```

```
  return (
```

```
    <ThemeContext.Provider value={contextValue}>
```

```
      {children}
```

```
    </ThemeContext.Provider>
```

```
  );
```

```
}
```

```
// =====
```

```
// 2. REDUCER LAYER: Multi-Stage Registration Machine
```

```
// =====
```

```
const initialState = {  
  step: 1,  
  formData: { username: "", email: "", role: "Developer" },  
  status: "idle", // 'idle' | 'submitting' | 'success'  
};
```

```
function registrationReducer(state, action) {  
  switch (action.type) {  
    case "FIELD_CHANGE":  
      return {  
        ...state,  
        formData: { ...state.formData, [action.field]: action.value },  
      };  
    case "NEXT_STEP":  
      return { ...state, step: state.step + 1 };  
    case "PREV_STEP":  
      return { ...state, step: Math.max(1, state.step - 1) };  
    case "SUBMIT_START":  
      return { ...state, status: "submitting" };  
    case "SUBMIT_SUCCESS":  
      return { ...state, status: "success", step: 3 };  
    case "RESET":  
      return initialState;  
    default:
```

```

    return state;
  }
}

// =====
// 3. PRESENTATIONAL VIEW LAYER
// =====

export default function MultiStageRegistration() {
  const { isDark, toggleTheme } = useContext(ThemeContext);
  const [state, dispatch] = useReducer(registrationReducer, initialFormState);

  const handleTextChange = (e) => {
    dispatch({
      type: "FIELD_CHANGE",
      field: e.target.name,
      value: e.target.value,
    });
  };

  const handleFinalSubmit = useCallback(async () => {
    dispatch({ type: "SUBMIT_START" });
    // Simulate async API registration call
    setTimeout(() => {
      dispatch({ type: "SUBMIT_SUCCESS" });
    }, 1200);
  }, []);

```

```

const styles = useMemo(
  () => ({
    background: isDark ? "#0F172A" : "#F8FAFC",
    color: isDark ? "#F8FAFC" : "#0F172A",
    padding: 24,
    borderRadius: 8,
    maxWidth: 480,
    margin: "40px auto",
    border: "1px solid #CBD5E1",
    fontFamily: "sans-serif",
  }),
  [isDark]
);

return (
  <div style={styles}>
    <header style={{ display: "flex", justifyContent: "space-between", marginBottom: 20 }}>
      <h2>Registration (Step {state.step} of 2)</h2>
      <button onClick={toggleTheme}>
        {isDark ? "☀️ Light Mode" : "🌙 Dark Mode"}
      </button>
    </header>

    {state.status === "submitting" && <p>Transmitting credentials to server...</p>}
  </div>
)

```

```
{state.step === 1 && state.status !== "submitting" && (
```

```
<section>
```

```
<div>
```

```
<label>Username: </label>
```

```
<input
```

```
  name="username"
```

```
  value={state.formData.username}
```

```
  onChange={handleTextChange}
```

```
  placeholder="e.g., alex_dev"
```

```
</div>
```

```
<button
```

```
  onClick={() => dispatch({ type: "NEXT_STEP" })}
```

```
  disabled={!state.formData.username.trim()}
```

```
  style={{ marginTop: 12 }}
```

```
>
```

```
  Continue →
```

```
</button>
```

```
</section>
```

```
)}
```

```
{state.step === 2 && state.status !== "submitting" && (
```

```
<section>
```

```
<div>
```

```
<label>Email: </label>
```

```
<input
```

```

      name="email"
      type="email"
      value={state.formData.email}
      onChange={handleTextChange}
      placeholder="alex@enterprise.com"
    />
  </div>
  <div style={{ marginTop: 12, display: "flex", gap: 8 }}>
    <button onClick={() => dispatch({ type: "PREV_STEP" })}>← Back</button>
    <button
      onClick={handleFinalSubmit}
      disabled={!state.formData.email.includes("@")}
    >
      Complete Registration
    </button>
  </div>
</section>
)}

{state.step === 3 && state.status === "success" && (
  <section style={{ textAlign: "center" }}>
    <h3 style={{ color: "#16A34A" }}>Profile Onboarded Successfully!</h3>
    <p>Welcome, {state.formData.username} {state.formData.email}</p>
    <button onClick={() => dispatch({ type: "RESET" })}>Register Another</button>
  </section>
)}

```

```
</div>  
  
);  
  
}
```

12. TECHNICAL INTERVIEW & RECAP CHEAT SHEET

High-Yield Questions & Answers

Q1: Why should you avoid using the Context API for high-frequency state updates?

Answer: When a Context Provider's value changes, every component calling `useContext` for that context will rerender, even if it only uses a piece of the context that didn't change. Unlike Redux with selectors, Context does not support granular component subscriptions out of the box, which can cause performance bottlenecks during frequent updates.

Q2: What is the primary difference between `useReducer` and Redux?

Answer: `useReducer` is a built-in React hook for managing complex state locally within a component or sub-tree. Redux is a full application-level state architecture featuring a centralized store, middleware pipelines, action interceptors, developer tooling, and global subscription optimization.

Q3: How does Redux Toolkit's `createSlice` allow writing "mutating" logic safely?

Answer: RTK integrates the **Immer** library. Inside reducers, Immer wraps the current state in a proxy object, tracks all mutations, and automatically produces an immutable copy with the updates applied, preventing direct state mutation bugs.

Q4: What is the purpose of `React.lazy` and `<Suspense>`?

Answer: They enable dynamic code splitting. `React.lazy` loads components asynchronously via dynamic imports when they are first rendered, while `<Suspense>` handles the loading state by displaying a fallback UI (such as a spinner) until the code bundle finishes downloading.

Q5: What role does a selector function play in Redux?

Answer: A selector reads and transforms specific pieces of state from the store. When used with `useSelector`, it subscribes the component *only* to changes in that specific slice of data, skipping rerenders when unrelated parts of the global store update.